

Blueberry

AI Dependency Validator

Project Status Report

Current state • what is complete • what could be improved
how to demonstrate it • what remains

Team	Parthiban M — 24BYB0135 Nisha Rathi — 24BYB0137
Repository	PhoenixKnight-in/Blueberry — branch feature/backend HEAD 19c6fea
Phase	Phase one — Python / PyPI only. Six-week build, weeks 1–6 delivered.
Report date	10 September 2026
Basis	Direct inspection of the working tree, plus every test suite, both linters and a live end-to-end run executed while writing this report. Nothing below is quoted from the README without being re-verified.

Project Status Report

Blueberry — AI Dependency Validator

1. Executive summary

Blueberry is **functionally complete against its own six-week plan**. All four planned components exist, are wired together, and work end to end against the live PyPI and GitHub APIs. The full test suite — 479 tests across three languages — passes, both TypeScript linters are clean, and a live pipeline run reproduces the exact three-row example table in the README. The system does the thing the project set out to do, and the code is of a noticeably higher standard than the brief required: the reasoning behind each design decision is written into the modules themselves rather than left implicit.

The gaps are not in the product. They are in **engineering process and deployment readiness**: no continuous integration, no database migrations, no authentication or rate limiting on the API, and — most urgently — five of the six commits that constitute weeks 2 through 6 exist only on one laptop. That is a single hard-drive failure away from losing everything except the week-one backend.

The one thing to fix today

main, dev and origin/feature/backend all point at 2f02a1b, the initial backend commit. The VS Code extension, the dashboard, the dashboard API and every README are unpushed local work. Run `git push -u origin feature/backend` before reading any further — everything else in this report can wait, and this cannot.

Status at a glance

Area	State	Assessment
Planned scope (weeks 1–6)	Complete	Backend, extension, dashboard, integration, docs all delivered.
Functional correctness	Verified	479/479 tests pass; live run scores requests 100, urllib4 75, requests 0.
Code quality	Strong	Clean module boundaries, pure testable cores, both linters clean, rationale documented in-place.
Design-document conformance	Good, 3 gaps	Two deviations already documented and defensible; three smaller ones are not yet written down (section 4).
Engineering process	Thin	No CI, no migrations, no Python lint config, no release packaging.
Production readiness	Not ready	No API auth, no server-side rate limiting, unbounded history growth.
Source control safety	At risk	5 commits unpushed; main is 5 commits behind.

2. Evidence this report is based on

Every claim below was re-established by running the project, not by reading its documentation. This matters for a status report: a README describes intent, and the gap between intent and behaviour is exactly what a status report exists to find. In this case there was no gap — the documentation is accurate — but that is a finding, not an assumption.

What was run	Result
backend: pytest	295 passed in 13.77 s — no network, no Redis, no Postgres (respx, fakeredis, SQLite)
extension: npm test	106 passed, 17 suites, 0.65 s — no editor required

What was run	Result
dashboard: npm test	78 passed , 4 files — jsdom with stubbed fetch
eslint (extension + dashboard)	Clean, no findings
Live pipeline, cache bypassed	requeusts → 100 / high_risk / 891 ms • urllib4 → 75 / high_risk / 685 ms • requests → 0 / safe / 1485 ms
Source inspection	~7,600 lines of application code and ~5,400 lines of test code read across all three components
git branch and remote state	5 commits ahead of origin; main 5 behind

The live run is worth dwelling on, because it demonstrates the project's central claim in three lines. `requeusts` scores 100 because it does not exist. `urllib4` scores 75 *although it does exist* — it is one character from `urllib3` and links no repository. `requests` scores 0 and says why: 54,292 stars and a commit two days ago. A registry-existence check alone catches only the first of the three.

3. What is built

Component	Week	App code	Test code	Tests	State
backend/ FastAPI service	1–2, 4	3,638	3,059	295	Complete
extension/ VS Code client	3	2,343	1,207	106	Complete
dashboard/ React SPA	4	1,592	1,106	78	Complete
Infrastructure Compose, 2 Dockerfiles, nginx	4	—	—	—	Complete
docs/ 3 PDFs + 4 READMEs	6	—	—	—	Complete
Total		7,573	5,372	479	

A test-to-code ratio of roughly 0.7:1 is high for a student project, and the tests are genuine unit tests rather than smoke tests — the risk engine alone has 501 lines of them. More importantly, **none of the 479 tests needs a network, a container or an editor**. That is an architectural property being reported by the test suite: the parser, the HTTP client, the cache and the scoring engine are all pure modules with the I/O pushed to their edges.

3.1 Backend — the verification service

Six HTTP endpoints across three routers, four verification checks, and a scoring engine that is the only place in the system permitted to decide what a fact is worth.

Endpoint	Purpose	Failure behaviour
POST /check	Verify one package, return a RiskReport	422 invalid name • 400 wrong ecosystem • 502 if PyPI unreachable
GET /health	Liveness probe used by Compose	—
GET /packages	Paginated, filterable check history	503 with an explanation, never an empty list
GET /packages/{id}	One stored check in full	404 unknown id • 503 if history unavailable
PATCH /packages/{id}	Triage: open / ignored / resolved	Score is never editable — only review state moves
GET /stats	KPI strip aggregates	503 if history unavailable

The four checks

Check	How it works	Can it fail?
PyPI	Registry existence plus metadata: first-release date, version count, author/maintainer attribution, project URLs. Redis-cached, 1 h.	Fatal. No lookup means no report — 502, never a <i>safe</i> verdict.
GitHub	Resolves the repository from PyPI's own metadata, then reads stars, last push, archived/fork state and name consistency. Redis-cached, 30 min, with backoff on rate limits.	Degrades. Unreachable scores exactly zero points with a note saying so.
Similarity	Levenshtein distance against a 423-name in-process corpus of high-traffic packages. No network, no I/O.	Cannot fail. Runs even when PyPI and GitHub are both down.
Risk engine	16 weighted rules, each producing a point value <i>and</i> a plain-English sentence. Pure function of its three inputs.	Cannot fail. No I/O anywhere in the module.

The **concurrency structure** is the payoff for choosing an async framework, and it is worth being able to explain precisely: PyPI and similarity are independent, so they run together under `asyncio.gather`. GitHub cannot join them, because the repository URL it inspects is read out of the PyPI response — a genuine data dependency, not an ordering preference. Running it in parallel would mean guessing the URL.

Security posture of the backend

- **Package names are anchored with `\A. . \Z`, not `^ . . $`.** In Python `$` also matches before a trailing newline, which would let a name carry a CRLF and inject a header into the outbound request.
- **Repository URLs are allowlisted and rebuilt, not sanitised.** The stored URL is reconstructed from a validated owner and repo, so nothing from the publisher-controlled original survives. A listing pointing at `http://169.254.169.254/` becomes "no repository", never a fetch — that is a closed SSRF path, deliberately.
- **The GitHub token has no default and is never written to a file in the repository.** It is read from the environment; a fine-grained token with no scopes is sufficient.
- **CORS is an explicit allowlist, not `*`,** because the dashboard issues state-changing PATCH requests.

3.2 VS Code extension — the thin client

The extension performs no verification of its own; it detects names, asks the backend, and renders the answer. That is what allows the scoring rules to change without shipping a new extension. It contributes 6 commands and 8 settings, and reads dependencies from four places: `import` statements, `requirements.txt` entries, `pip install` commands in comments or notebook cells, and `pyproject.toml` dependency lists.

The parser is a line scanner rather than an AST walker, and that is the right call: an AST needs the file to be syntactically valid, and the moment the tool most wants to speak up is while a completion is still being typed, when it usually is not. Line scanning degrades to "finds slightly less"; an AST degrades to "finds nothing".

Three filters protect the false-positive budget, which is the resource a security extension actually lives on — a wall of wrong warnings is how it gets uninstalled. The standard library is excluded (`import os` is not a package), an import-name to distribution-name map is applied (you `import yaml` but install PyYAML), and relative imports are skipped. On the scoring side, an exact hit in the popular-package corpus can never be a typosquat — the corpus doubles as a whitelist — and names of four characters or fewer match only at distance 1.

3.3 React dashboard — the review surface

Thirteen components, and a strict role: it is a **pure reader**. It never triggers a verification, so nothing a reviewer does can cost the developer in the editor a round trip or a slot in the GitHub rate-limit budget. It shows a KPI strip (total checks, open flags, hallucinations, typosquats), a severity distribution, four filters, a paginated table, and a detail view that reproduces the score rule by rule.

The single best design decision in the project

Nothing is allowed to fail open. A check that does not complete is reported as "could not verify", never as silence — because in a security tool silence reads as an all-clear, and the entire premise is that a developer is about to trust something they should not. The rule is enforced at every layer: PyPI failure returns 502; unreachable GitHub scores zero points with a note; the extension client has no code path that turns a timeout into a verdict; and the dashboard renders an error banner rather than an empty table, because "nothing was flagged" and "we could not ask" are opposite conclusions.

4. Where the build departs from the design document

An examiner with the Final Design Document open will compare it against the code. Two deviations are already written up in the root README and are defensible. Three smaller ones are not yet written down anywhere, and should be — an undocumented deviation reads as an oversight, whereas a documented one reads as a decision.

Design document says	Code does	Status
Eight normalised tables (packages, package_metadata, repository_health, similarity_results, risk_scores, flagged_packages, ...)	One denormalised check_results table with four indexes: flat columns for what the dashboard filters and sorts on, JSON blobs for what it only renders.	Documented Every check writes all of them at once and the dashboard reads them back together, so the normalised form buys five joins and no extra information.
popular_packages and extension_settings as database tables	The corpus is an in-process Python list; settings live in VS Code's own settings store.	Documented The list is what lets the similarity check run with no network at all — it cannot rate-limit or time out, so requests is still flagged during a full outage.
package_metadata.download_count as a trust signal	Not implemented — no download data is read or scored.	Undocumented gap Defensible: the PyPI JSON API does not expose download counts at all. It needs a separate source (pypistats or the BigQuery dataset). Write this down.
Editor Event Listener consumes "inline completions, AI chat responses, terminal text "	Document events only: edit, open, save, focus change. Terminal contents are never read.	Undocumented gap Defensible: VS Code has no stable API for reading terminal output. The parser does catch pip install written in a comment or notebook cell, which is the nearest supported equivalent. Say so explicitly.
Class diagram names PackageCandidate, CheckRequest, RiskScore, FlaggedPackage as backend classes	The backend has CheckRequest and RiskReport/RiskAssessment; PackageCandidate lives in the extension, and FlaggedPackage is the status column.	Cosmetic Same concepts, different names and placement. Worth one sentence in a mapping table so nobody has to guess.

5. What could be improved

Ordered by return on the time invested. The first four are the ones that change how the project is assessed; the rest are polish. None of them is a defect in what already works.

5.1 Highest return — do these first

#	Improvement	Effort	Priority	Why it matters
1	Push everything and merge to main	5 min	Critical	Five commits — the extension, the dashboard, the dashboard API, the READMEs — exist on one machine only. main still shows a bare backend. Anyone who clones the repository today sees roughly a sixth of the project. This is both the largest risk and the cheapest fix in the report.
2	Add continuous integration	1 hr	High	There is no <code>.github/workflows/</code> . 479 tests exist and pass, but only when someone remembers to run them. A single workflow running all three suites plus both linters on push turns "we tested it" into a green tick anyone can verify, and it is the standard evidence of an iterative process — which is the process model the project document commits to.
3	Generate the Alembic migration that is already paid for	1–2 hr	High	<code>alembic</code> is declared in <code>requirements.txt</code> , but there is no <code>alembic/</code> directory and no migration. The schema is created at startup by <code>database_auto_create=True</code> , which is fine for a demo and cannot evolve a live database. Right now the dependency reads as an intention rather than a capability.
4	Authenticate and rate-limit the API	2 hr	High	Anything that can reach port 8000 can spend the GitHub token's 5,000 requests/hour and write unbounded rows into <code>check_results</code> . For a local demo this is irrelevant; for anything deployed it is the first thing a reviewer will ask about. A shared API key plus a per-IP limit closes it.
5	Add POST /check/batch	3 hr	Medium	A 40-line <code>requirements.txt</code> currently costs 40 HTTP requests. The extension already limits concurrency, so nothing breaks — but one request returning 40 reports would cut latency sharply and reduce rate-limit pressure. It also demonstrates better than anything else that the pipeline is genuinely async.

5.2 Worth doing — polish and hardening

#	Improvement	Effort	Priority	Why it matters
6	Grow the popular-package corpus beyond 423 names	2 hr	Medium	The corpus is hand-listed. A typosquat of a package ranked, say, 600 by downloads is invisible to the similarity check, and the corpus also serves as the whitelist, so its size sets the ceiling on both recall and precision. Generating it from a published top-N list at build time makes coverage a build artefact rather than a maintenance chore.
7	Add a Python lint and format configuration	30 min	Medium	Both TypeScript projects have ESLint and Prettier; the backend has neither a <code>ruff</code> nor a <code>black</code> configuration. The code is already consistent, so this is about being able to prove it — and the asymmetry is visible to anyone comparing the three <code>package.json/requirements</code> files.
8	Clean the noisy test output	30 min	Low	<code>App.test.tsx</code> emits <code>React act(...)</code> warnings and the backend suite carries a Starlette <code>TestClient</code> deprecation warning. All 479 tests pass, but if the suites are run live in a demo, warnings scrolling past undercut the result.

#	Improvement	Effort	Priority	Why it matters
9	Add request IDs and structured logging	1–2 hr	Low	duration_ms is recorded per report, but there is no way to attribute a slow check to PyPI versus GitHub versus a cache miss. Per-stage timings would make a performance question answerable on the spot instead of by guesswork.
10	Decide what happens to old history	1 hr	Low	check_results grows without bound — one row per check, forever, and the extension checks on every save. A retention window, or collapsing repeats of the same package into a single row with a counter, keeps the dashboard readable.

6. What is left to be done

Three distinct buckets, and it is worth keeping them apart. Only the first is actually outstanding work against the current phase; the second is scope the project explicitly deferred, and the third only becomes relevant if Blueberry is ever deployed rather than demonstrated.

6.1 Outstanding for this phase — a short list

Task	Effort	Status
Push feature/backend; merge into dev and main	5 min	Not started
Rehearse the demonstration end to end at least twice, timed	1 hr	Not started
Record a fallback screen capture of the full demo	30 min	Not started
CI workflow running all three suites and both linters	1 hr	Not started
Document the three undocumented design deviations (section 4)	20 min	Not started
Alembic initial migration	1–2 hr	Dependency declared, unused
Package the extension as a .vsix (@vscode/vsce is not a dev dependency)	30 min	F5 dev host works; no installable build

That is roughly a day of work, and none of it is product development. The phase-one product itself is finished.

6.2 Declared future work — deliberately out of scope

These are named as future work in both the project guide and the design document. They are not gaps, and should be presented as a roadmap rather than as an apology.

- **Multi-registry support** — npm, crates.io, Maven, NuGet. The architecture already anticipates it: ecosystem is a field on the request, the response and the database row, and POST /check returns a specific 400 for anything other than pypi. The work is one checker module and one corpus per registry, not a redesign.
- **ML-based hallucination detection**, complementing rather than replacing the rules. The rule engine stays as the explanation layer — which is why it was chosen over a model in the first place.
- **CI/CD integration** so checks run in a build pipeline rather than only at suggestion time. This is the smallest of the four and arguably the highest value: a GitHub Action that fails a build on a high-risk dependency needs only the existing POST /check.
- **Enterprise dashboard** for teams tracking dependency risk across many repositories. Needs the multi-tenancy the current single denormalised table does not have.

6.3 Production readiness — if it were ever deployed

Worth having an answer ready for, because "what would you need to change to run this for real?" is a natural question and the honest answer is short: authentication and server-side rate limiting on the API; Alembic migrations instead of `create_all` at startup; a retention policy for the check history; structured logs and per-stage metrics; a signed, published extension build; and secret management for the GitHub token beyond an environment variable. Notably, the things that are usually hardest to retrofit — input validation, SSRF defence, failure semantics and test coverage — are already in place.

7. How to demonstrate it effectively

The principle behind the running order

Demonstrate the three failure modes in order of increasing subtlety, and never show a score without showing the reason underneath it. The project's thesis is not "we can detect bad packages" — a registry lookup does that. The thesis is **explainability**: a developer who cannot reconstruct a warning will dismiss it. So every screen in the demo should answer "why?" before anyone asks.

7.1 Pre-flight, 15 minutes before

#	Step	Why it matters
1	<code>export BLUEBERRY_GITHUB_TOKEN=...</code>	The single most likely thing to break a live demo. Unauthenticated GitHub allows 60 requests/hour per IP. Without a token, repository checks start reporting "unavailable" part-way through and requests stops looking healthy.
2	<code>docker compose up --build</code> , wait for all four health checks	Compose gates the API on Redis and Postgres being healthy, and the dashboard on the API. Starting early means the demo starts on a warm stack.
3	Pre-warm the cache: check <code>requests</code> , <code>urllib4</code> , <code>requests</code> , <code>numpy</code> once each	The best insurance available. PyPI results cache for 1 hour and GitHub for 30 minutes as complete objects, so a pre-warmed name is served entirely from Redis. If the venue's network fails, the demo still runs.
4	Seed the dashboard with a dozen checks across all three severities	An empty table is a poor first impression, and the KPI strip and severity distribution only make sense with data behind them.
5	Open three windows: the Extension Development Host (F5) with a scratch <code>.py</code> file, the dashboard, and <code>/docs</code>	Alt-tabbing between prepared windows is invisible; opening applications live is not.
6	Increase the editor font size and pick a high-contrast theme	The demo turns on a squiggly underline and a hover card being legible from the back of a room.

7.2 The seven-minute run

Time	Beat	What to do and what to say
0:00	The problem, in the editor	Type <code>import requests</code> into a Python file. Say nothing — let the underline appear on its own, then hover. Score 100: <i>not found on PyPI, closest real package is requests</i> . The line to deliver: today <code>pip</code> would error on this — but an attacker who has registered the hallucinated name turns that error into a silent install. That is slopsquatting, and it is why the check runs before the terminal, not after.
0:45	The case a registry check cannot catch	Type <code>urllib4</code> . It exists. <code>pip install urllib4</code> succeeds without complaint. Blueberry scores it 75: one character from <code>urllib3</code> , and it links no repository. This is the strongest technical moment in the demo — existence alone proves nothing, and this is the case that justifies the whole similarity engine.

Time	Beat	What to do and what to say
1:45	The warnings that deliberately do not fire	Type <code>import os, import yaml, from .models import X</code> . Nothing happens, and that is the point: the standard library is filtered, <code>yaml</code> is mapped to its distribution name <code>PyYAML</code> before checking, and relative imports are skipped. Say it plainly: a security extension that cries wolf gets uninstalled, so the false-positive budget is spent deliberately.
2:30	The score is reconstructable by hand	Switch to the dashboard, open <code>urllib4</code> . Walk the signal list: 60 for distance-1 similarity, 15 for no repository link, total 75. Then the sentence that lands: <code>sum(signal.points) == final_score</code> is asserted in the test suite. That is the whole argument for rule-based over machine-learned scoring — not accuracy, auditability.
3:30	It never fails open	Run <code>docker compose stop api</code> and check a package. The status bar reads Blueberry: offline rather than going quiet; refresh the dashboard and it shows an error banner rather than an empty table. Say why: silence reads as an all-clear, and "nothing was flagged" and "we could not ask" are opposite conclusions. Restart the container.
4:15	The architecture, in one API call	Open <code>/docs</code> and POST requests. Point at <code>duration_ms</code> — around 1,400 ms cold. Explain the shape: PyPI and similarity run concurrently, GitHub must follow because it reads the repository URL out of PyPI's response. Then send it again: the second call is far faster, which is Redis doing its job and the reason the GitHub budget survives a room full of developers.
5:15	Triage, and what stays immutable	Mark <code>urllib4</code> as <i>ignored</i> . The row moves, the KPI strip updates, the filter picks it up — and the score does not change. Only the review state moves, so the original verdict stays auditable next to the human decision made about it.
6:15	Close on the tests	Run all three suites live. 479 tests, none of which needs a network, a container or an editor . Say why that is an architectural claim rather than a testing one: the parser, the client, the cache and the risk engine are pure modules with the I/O at their edges, so the parts that get edited most often are the cheapest to verify.

7.3 What to do when something breaks

If this happens	Do this
GitHub rate-limits mid-demo	Demonstrate it rather than apologising for it. The report still returns, with a zero-point note reading "GitHub could not be reached". That is the degradation story working in public: not risky, not safe, just honestly unknown. Few demos get to show their failure path on purpose.
The network dies completely	Pre-warmed names still resolve from Redis. Stay on requests, <code>urllib4</code> and <code>requests</code> — and do not check a package you have not warmed, because an uncached lookup returns 502 by design.
The extension host misbehaves	Fall back to <code>/docs</code> and drive POST <code>/check</code> directly. Every beat except the editor interaction survives, and the JSON shows more, not less.
Docker will not start	Run <code>uvicorn app.main:app --reload</code> and <code>npm run dev</code> . Redis and Postgres are optional — the service logs a warning, skips caching and history, and every check still runs. Only the dashboard beat is lost.
You run out of time	Cut beats 6 and 7. Never cut beat 2 (<code>urllib4</code>) or beat 4 (the score breakdown) — those two carry the argument.

7.4 Questions to expect, and where the answer already lives

Each of these has an answer in the code already. Knowing where it is turns a defensive moment into a strong one.

Question	Answer
"Why rule-based rather than machine learning?"	Explainability is the product, not a constraint on it. A developer who sees "risk: 74" dismisses it; one who sees "no repository, 92% similar to requests" acts. Also, honestly: there is no labelled corpus of hallucinated package names to train on.
"Your design document has eight tables and your code has one."	A documented, deliberate departure — it is in the README. Every check writes all eight at once and the dashboard reads them back together, so normalising buys five joins per row and no extra information. The status column carries <code>flagged_packages.status</code> .
"What stops a malicious PyPI listing from pointing your server at an internal address?"	Repository URLs are allowlisted, not blocklisted, and the stored URL is rebuilt from a validated owner and repo, so nothing from the original string survives. A Source field pointing at <code>169.254.169.254</code> becomes "no repository", never a fetch.
"Why <code>\A.. \Z</code> and not <code>^\.\$</code> in the name regex?"	In Python <code>\$</code> also matches immediately before a trailing newline, so <code>^\.\$</code> would accept a name ending in <code>\n</code> — and that name goes into an outbound HTTP request. It is a CRLF header-injection path, closed at validation.
"What happens when GitHub is down?"	Exactly zero points, plus a note saying the check could not run. Scoring it as risky would flag half the ecosystem during an outage; scoring it as healthy is a false negative, which is worse than no check at all because the developer believes something was verified.
"Is the extension doing the checking?"	No — it detects names and renders answers, nothing else. That is what allows the scoring rules to be retuned without shipping a new extension, and it keeps every scoring decision in one place.
"How would you add npm?"	One checker module and one corpus. <code>ecosystem</code> is already a field on the request, the report and the stored row, and <code>POST /check</code> already returns a specific 400 for anything other than <code>pypi</code> .
"How did you pick the weights?"	Iteratively, against real package names — which is the reason the process model is iterative-incremental. They live in <code>config.RiskWeights</code> and are environment-overridable, so retuning is a configuration change: <code>BLUEBERRY_RISK__TYPOSQUAT_DISTANCE_1=70</code> .

8. Risk register

Risk	Likelihood	Impact	Mitigation
Unpushed work lost with the machine	Low	Severe	Push today. Five commits — five of the six weeks — exist in one place.
GitHub token unset at demo time	High	High	Set it and verify with one live check before the audience arrives; pre-warm the cache.
Venue network unavailable	Medium	High	Pre-warm every package used in the demo; Redis serves them without any network.
PyPI slow or unreachable	Low	Medium	By design an uncached check returns 502 rather than a false safe. Pre-warming is the only real mitigation — and the 502 itself is explainable.
Examiner compares design document to code	High	Medium	Two deviations are already documented; write up the three in section 4 before submission.
Extension host fails to launch	Low	Medium	Drive <code>/docs</code> instead; keep a recorded capture as a last resort.

9. Conclusion

Blueberry has delivered what it set out to deliver, and the parts that are hardest to add later are the parts that are already there: clear module boundaries, pure and independently testable cores, honest failure semantics, and validation of untrusted input at the point it enters the system. The verification claims in the documentation hold up when the code is actually run, which is not something that can be assumed.

What is missing is process scaffolding rather than product: continuous integration, migrations, and a lint configuration on the Python side. Those are hours of work, not weeks. The one genuinely urgent item is source-control safety — five of the six weeks of this project currently exist on a single machine, and that should be resolved before anything else in this report is acted on.

Recommended order of work

1. Push and merge (5 minutes). **2.** Add CI (1 hour). **3.** Document the three design deviations (20 minutes). **4.** Rehearse the demo twice, timed, including the network-failure fallback (1 hour). **5.** Everything else in section 5, in the order listed.